Foundation Certificate in Higher Education

INFORMATICS INSTITUTE OF TECHNOLOGY

Individual Coursework

Module Code	: DOC 334
Module Title	: Computer Programming
Module Leader	: Nishan Saliya Harankahawa
Assessment Type	: Individual Course work
Issued Date	: 10th November 2025
Hand-in Date	: 30th November 2025
Weight	: 30%

Student ID : 20250202

Student Name : Nonim Helika Rajasuriya

Centre : Colombo

The department is not responsible if an assignment is lost. To cover this eventuality, you are advised to take a copy of the assignment OR to ensure you have the means of re-creating it.

.

Exceptional Factors Affecting Your Performance:

Students should submit a Mitigating Circumstances Form with evidence, if they miss the submission deadline. This form must be submitted within five working days of the submission date. The Mitigating Circumstances Form is available on the LMS, in the General Information and Policies section.

Acknowledgement

I would like to express my sincere gratitude to the module leader, Mr. Nishan Saliya Harankahawa, for providing clear guidance, well-structured learning materials, and continuous support throughout this coursework. The instructions and examples given in the assignment brief were invaluable in helping me understand the requirements and successfully develop the program.

furthermore, I would like to heartfelt appreciation for Ms. Tharushi Amarasinghe who supported for support ,guidance, and encouragements which have facilitated my progress and enchanted my work at each stage and also I would like to thank for my batchmates who are supported during the development of the project.

Finally, I appreciate the learning environment, facilities, and technical support made available by the institution, which enabled me to complete this assignment effectively.

Table of Contents

Contents

Acknowledgement 2

Table of Contents 3

Table of figures 5

Game flow step by step 6

Folder Structure 7

code 7

gundown.py 7

user_validation.py 9

hidden_obj_placement.py 11

hidden _location_genarator.py 12

matrix.py 12

final_matrix.py 13

Runing Program 15

Test Cases 19

Test Case 1 19

Test Case 2 20

Test Case 3 20

Test Case 4 21

Test Case 5 21

Test Case 6 22

Test Case 7 22

Test Case 8 23

Test Case 9 23

Test Case 10 24

Test Case 11 24

Test Case 12 25
Test Case 13 25
Test Case 14 26
Test Case 15 26
Test Case 16 27
Test Case 17 28
Test Case 18 28
Test Case 19 29
Test Case Table 30
References 33

Table of figures

Figure 7 display the help page 15
Figure 8 Display result in normal mode 16
Figure 9 Display result in cheat mode 17
Figure 10 Display save files according to the mode 18
Figure 11 Save values in text file in normal mode 18
Figure 12 Save values in text file in cheat mode 18
Figure 13 Test Case 1 19
Figure 14 Test Case 2 20
Figure 15 Test Case 3 20
Figure 16 Test Case 4 21
Figure 17 Test Case 5 21
Figure 18 Test Case 6 22
Figure 19 Test Case 7 22
Figure 20 Test Case 8 23
Figure 21 Test Case 9 23
Figure 22 Test Case 10 24
Figure 23 Test Case 11 24
Figure 24 Test Case 12 25
Figure 25 Test Case 13 25
Figure 26 Test Case 14 26
Figure 27 Test Case 15 27
Figure 28 Test Case 16 27
Figure 29 Test Case 17 28
Figure 30 Test Case 18 28
Figure 31 Test Case 19 29

Game flow step by step

1. **Start:** The player runs a command like gundown 3x3 3 to choose grid size and hidden object count. The grid must be between 3x3 and 5x5, and hidden objects must be valid per rules. A help command (gundown help) displays how to play.
2. **Validate input:** The program verifies the order of the matrix and object limits (e.g., 5x6 is invalid, 5x5 cannot hold 13 objects). If invalid, it displays an error and exits.
3. **Generate grid:** The program builds the grid and labels cells with running numbers (row-major order), internally placing hidden objects at random positions without revealing them.
4. **Prompt guesses:** The player is asked to enter each guess by cell number—exactly as many guesses as the hidden object count (e.g., "Guess hidden object location 1 of 3:").
5. **Reveal results:** After all guesses, the program shows the grid with hits and misses marked distinctly (e.g., X for hits, H for unseen hidden objects), then prints a text summary: hidden locations, found locations, and the score for example, "1 out of 3 found (33%)."
6. **Persist to file:** The program saves one text file per play, containing date, time, hidden/found locations, and the summary. The filename format is "YYYYMMDD_ HHMM_ RRRR.txt," where RRRR is a random 4-digit code.
7. **Cheat mode (optional):** If the player runs gundown 3x3 3 show, the initial grid displays blank spaces at hidden locations. The saved filename appends with a "C" .txt."

8.Outputs and saving:

- **Date and Time**: Date and time according to the real time format
- **Visual grid:** Numbers for cells, with post-play markers distinguishing hits and missed hidden objects. This gives immediate feedback on the player's accuracy.
- **Text summary:** Lists hidden and found locations plus a final performance line, for example: "1 out of 3 found (33%)."
- **File record:** Exactly one .TXT file per game, the summary. Filenames include date, time, and a random code, with "C" appended if cheat mode was used.

Folder Structure

1. Main Program
    a. gundown.py
    b. game files(folder)
        i. final_matrix.py
        ii. hidden_location_genaratoe.py
        iii. hidden_obj_placement.py
        iv. matrix.py
        v. user_validation.py
        vi. ende_game_banner.txt
        vii. Game_banner.txt
        viii. help.txt

code

gundown.py

```
import sys
import random
import datetime
from game_files  import matrix
from game_files import user_validation
from game_files import hidden_obj_placement
from game_files import final_matrix

def final_results(hidden_locations,found_objects):
  print("Final Results")
  print("Hidden locations : ", *hidden_locations,sep=',')#unpack and add a comma in
between the values
  print("Found objects   : ", *found_objects,sep=',')#unpack and add a comma in between
the values
  print(f"{len(found_objects)} of {len(hidden_locations)} found
({len(found_objects)/len(hidden_locations)*100:.0f}%)")
  now = datetime.datetime.now()             # Get the current local date and time from the
system clock
```

```
  date_str = now.strftime("%Y%m%d")          # Get the current local date from the system
clock separately
  time_str = now.strftime("%H%M")            # Get the current local time from the system
clock separately
  random_code = random.randint(1000, 9999)     # creating a random 4 digit number for the
text file name
  if mode == True:
    filename = str(str(date_str) + "_" + str(time_str) + "_" + str(random_code) + "C.txt")  # if it
is normal mode then just the filename with "C"
  else:
    filename = str(str(date_str) + "_" + str(time_str) + "_" + str(random_code) + ".txt")  # if it is
normal mode then just the filename without "C"
  fo=None
  fo=open(filename, "w")                # open the file in write mode
  fo.write("Date : " + str(now.strftime("%Y-%m-%d")))  # writing the current date in the text
file
  fo.write("\n")# writing a new line in the text file
  fo.write("Time : " + str(now.strftime("%H:%M")))  # writing the current time in the text file
  fo.write("\n")# writing a new line in the text file
  fo.write("Hidden locations : "+ str(hidden_locations ))
  fo.write("\n")# writing a new line in the text file
  fo.write("Found objects    : "+ str(found_objects ))
  fo.write("\n")# writing a new line in the text file
  fo.write(f"{len(found_objects)}  of  {len(hidden_locations)} found
({len(found_objects)/len(hidden_locations)*100:.0f}%)")
  fo.close()
  print()
  end=None #end game banner variable
  end=open("game_files//end_game_banner.txt","r")
  print(end.read())# print the end game banner
  end.close()# closing the end game banner text file
  return
```

```
# Main flow

mode=user_validation.input_validation()
no_rows ,no_column=user_validation.grid_validation()
hidden_count,cell_count=user_validation.hidden_obj_validation(no_rows
,no_column)
hidden_locations=hidden_obj_placement.find_hidden_objects(hidden_count,cell_count)
matrix.matrix_generation(no_rows, no_column,mode,hidden_locations)
guess_positions=hidden_obj_placement.hidden_object_placement(hidden_count,no_row
s,no_column)
found_objects, missed_objects=final_matrix.check_found_missed(guess_positions,
hidden_locations, no_rows, no_column
final_results(hidden_locations,found_objects)
```

user_validation.py

```
import sys
def input_validation():
    help = None  # help variable
    mode = False  # mode variable
    banner = None  # banner variable
    # Display help information
    if len(sys.argv) == 2 and sys.argv[1].lower() == "help":
        print()
        help = open("game_files//help.txt", "r")  # open the help text file
        print(help.read())  # print the help information
        help.close()  # closing the help text file
        print()
        sys.exit()
    elif len(sys.argv) == 3:
        print()
        # open the game banner text file
        banner = open("game_files//game_banner.txt", "r")
        print(banner.read())  # print the game banner
        banner.close()  # closing the game banner text file
        print()
        return
```

```
    # enable the cheat mode
    elif len(sys.argv) == 4 and sys.argv[3].lower() == "show":
        mode = True  # set mode to True for cheat mode
        print()
        # open the game banner text file
        banner = open("game_files//game_banner.txt", "r")
        print(banner.read())  # print the game banner
        banner.close()  # closing the game banner text file
        print()
        return mode  # return mode for cheat mode
    else:
        # Invalid number of arguments
        print("Only two arguments allowed ex: gundown help or gundown 3x3 4")
        sys.exit()
    return

def grid_validation():
    try:
        grid_order = []  # list to store grid order
        # split the grid size input based on 'x'
        grid_order = sys.argv[1].split('x')
        # convert the first part to integer for rows
        no_rows = int(grid_order[0])
        # convert the second part to integer for columns
        no_column = int(grid_order[1])
        if no_rows >= 3 and no_column <= 5:
            return no_rows, no_column
        else:
            print("Invalid grid size !!! Rows should be >=3 and Columns should be <=5 ")
            sys.exit()
    except ValueError:  # exception handling for invalid input
        print("Invalid grid size format !!! Use Integers RowsxColumns (e.g., 3x4)")
        sys.exit()
    return

def hidden_obj_validation(no_rows, no_column):
    hidden_count = 0  # hidden count initialization
    try:
```

```
    # convert the hidden object count input to integer
    hidden_count = int(sys.argv[2])
    cell_count = no_rows * no_column  # calculate total cell count
    # validating hidden object count
    if 2 <= hidden_count <= round(cell_count/2) : # validate the hidden object count with
the user input
      return hidden_count, cell_count  # return hidden object count and cell count
    else:
      print(f"Invalid hidden objects count !!! It should be  2 or {round(cell_count/2)}")
      sys.exit()
  except ValueError:  # exception handling for invalid input
    print("Invalid hidden objects count !!! It should be an Integer")
    sys.exit()

  return
```

hidden_obj_placement.py

```
def find_hidden_objects(hidden_count,cell_count):
 hidden_locations =[]
 hidden_locations= random.sample(range(1,cell_count),hidden_count)
 return hidden_locations                                    object locations
def hidden_object_placement(hidden_count,no_rows,no_column
  guess_positions =[]
  temp_position =0
  guess_count=0
  while guess_count < hidden_count:
   guess_count +=1
   try:
     temp_position = int(input(f"Guess hidden object location {guess_count} of
{hidden_count} : "))
     if temp_position not in guess_positions and 1 <= temp_position <= no_rows *
no_column:
       guess_positions.append(temp_position)
     else:
       print("Duplicate position entered or value is not in the column range !!! Please enter
a different position")
       guess_count -=1
   except :
```

```
        print("Invalid input !!! Please enter an Integer value for
position")                    #exception message
        temp_position = int(input(f"Guess hidden object location {guess_count} of
{hidden_count} : "))#get user input
    print()
    return guess_positions
```

hidden _location_genarator.py

```
import random

def find_hidden_objects(hidden_count, cell_count):
    hidden_locations = []
    hidden_locations = random.sample(range(1, cell_count), hidden_count)
    return hidden_locations
```

matrix.py

```
def matrix_generation(no_rows ,no_column,mode,hidden_locations):
    if mode == True: #cheat mode enabled
        # Initialize counter
        counter = 1

        # Create the horizontal separator
        separator = "+---" * no_column + "+"

        # Generate and print the grid
        for r in range(no_rows):
            print(separator)  # Print separator before each row
            row_items = [] # Generate and print the grid
            for c in range(no_column): # Generate and print the grid
                if counter in hidden_locations: # Generate and print the grid
                    row_items.append("|  ")  # Hidden object location
                else:
                    row_items.append(f"|{counter:2}")
                counter += 1                #counter increment
```

```
        print(" ".join(row_items) + " |")  # Print the row
    print(separator)  # Final bottom border
    print() # Print a new line
  else:
    # Initialize counter
    counter = 1

    # Create the horizontal separator
    separator = "+---" * no_column + "+" #  Create the horizontal separator

    # Generate and print the grid
    for r in range(no_rows):
      print(separator)  # Print separator before each row
      row_items = []
      for c in range(no_column):
        row_items.append(f"|{counter:2}")
        counter += 1
      print(" ".join(row_items) + " |")
    print(separator)  # Final bottom border
    print() #   Print a new line
  return
```

final_matrix.py

```
def check_found_missed(guess_positions, hidden_locations, no_rows, no_column):
  found_objects = list(set(guess_positions) & set(hidden_locations))
  missed_objects = list(set(hidden_locations) - set(guess_positions))
  # Initialize counter
  counter = 1
  # Create the horizontal separator
  separator = "+---" * no_column + "+"

  # Generate and print the grid
  for r in range(no_rows):
    print(separator)
    row_items = []
    for c in range(no_column):
```

```
        if counter in found_objects:
            row_items.append("| X")  # Found object
        elif counter in missed_objects:
            row_items.append("| H")  # Missed object
        else:
            row_items.append(f"|{counter:2}")  # Normal number
        counter += 1
    print(" ".join(row_items) + " |")
    print(separator)  # Final bottom border
    print()
    return found_objects, missed_objects
```

Runing Program

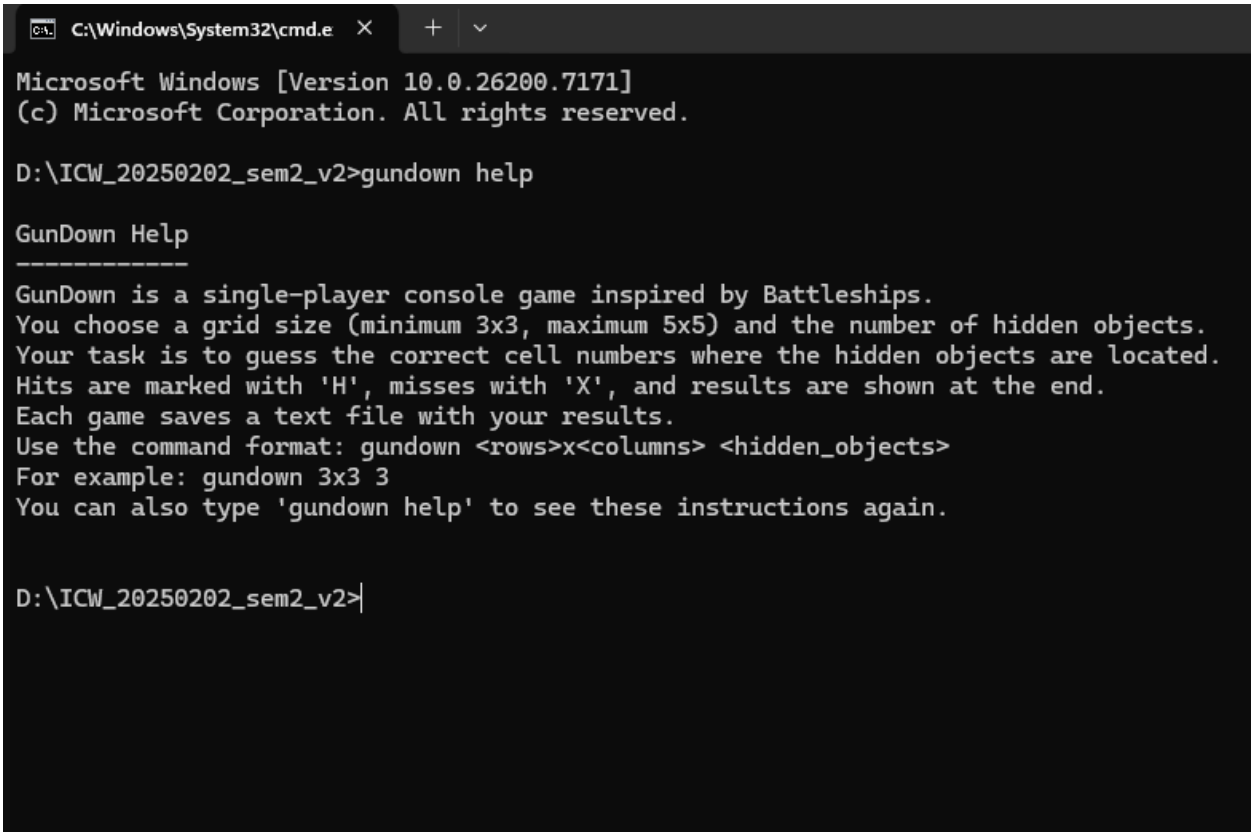


Figure 1 display the help page



Figure 2 Display result in normal mode



Figure 3 Display result in cheat mode

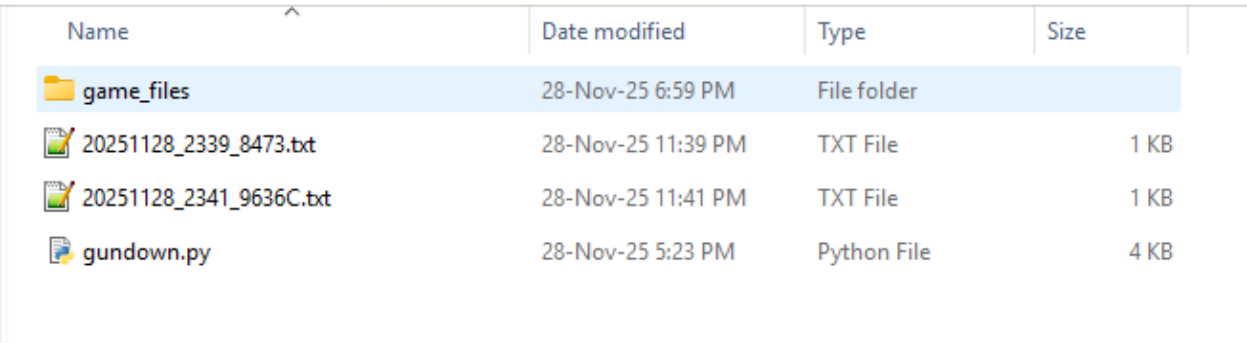


Figure 4 Display save files according to the mode

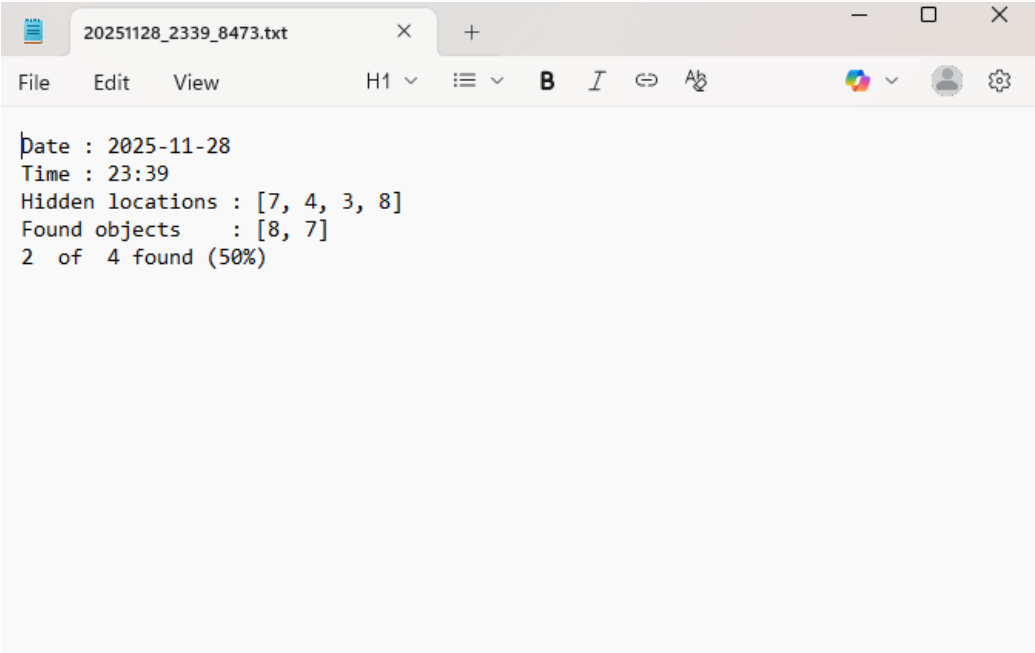


Figure 5 Save values in text file in normal mode

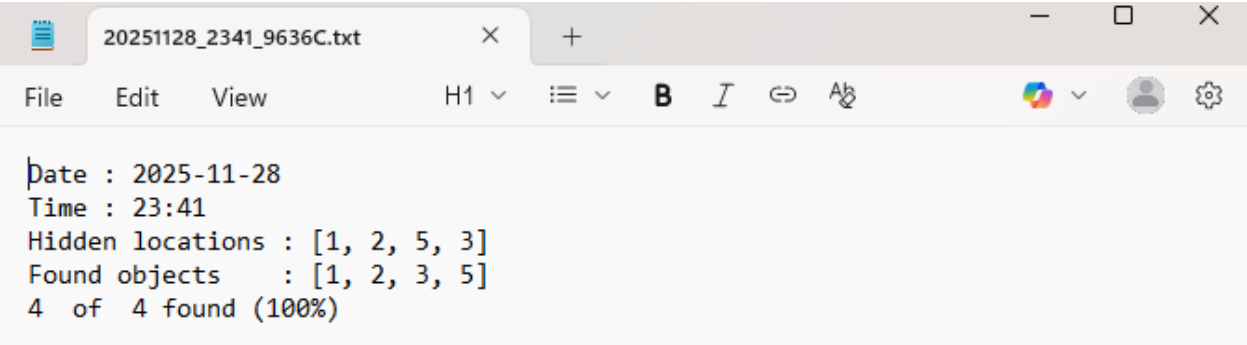


Figure 6 Save values in text file in cheat mode

Test Cases

Test Case 1

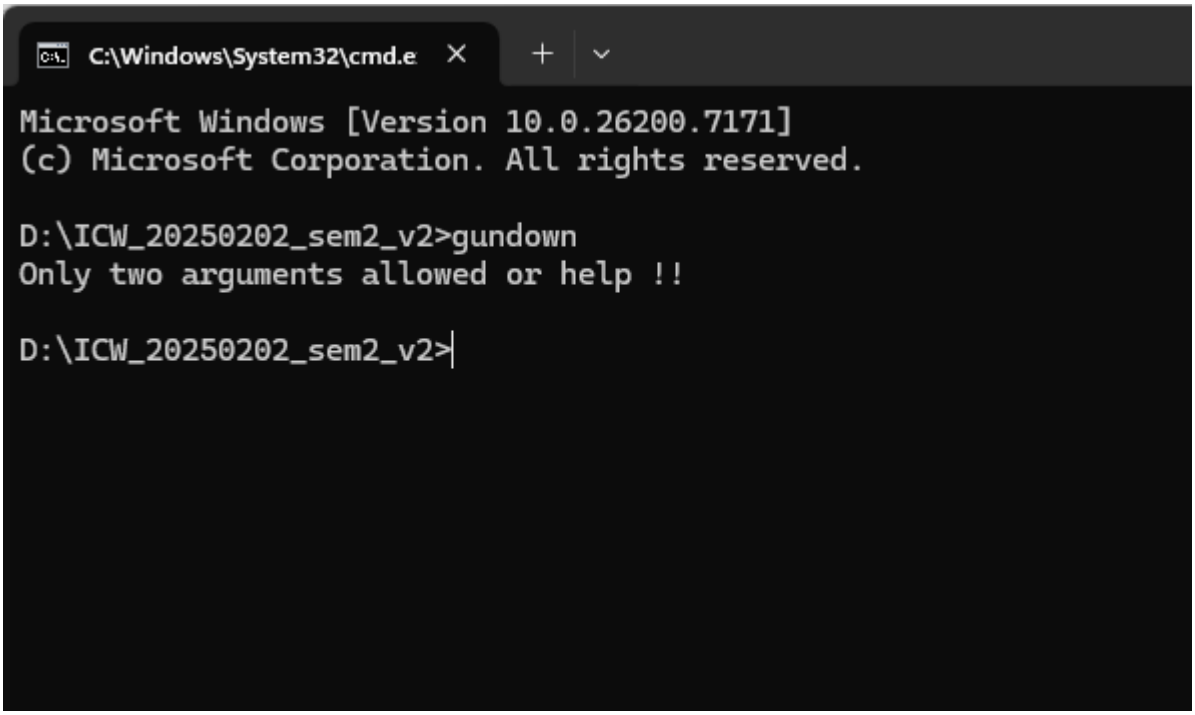


Figure 7 Test Case 1

Test Case 2

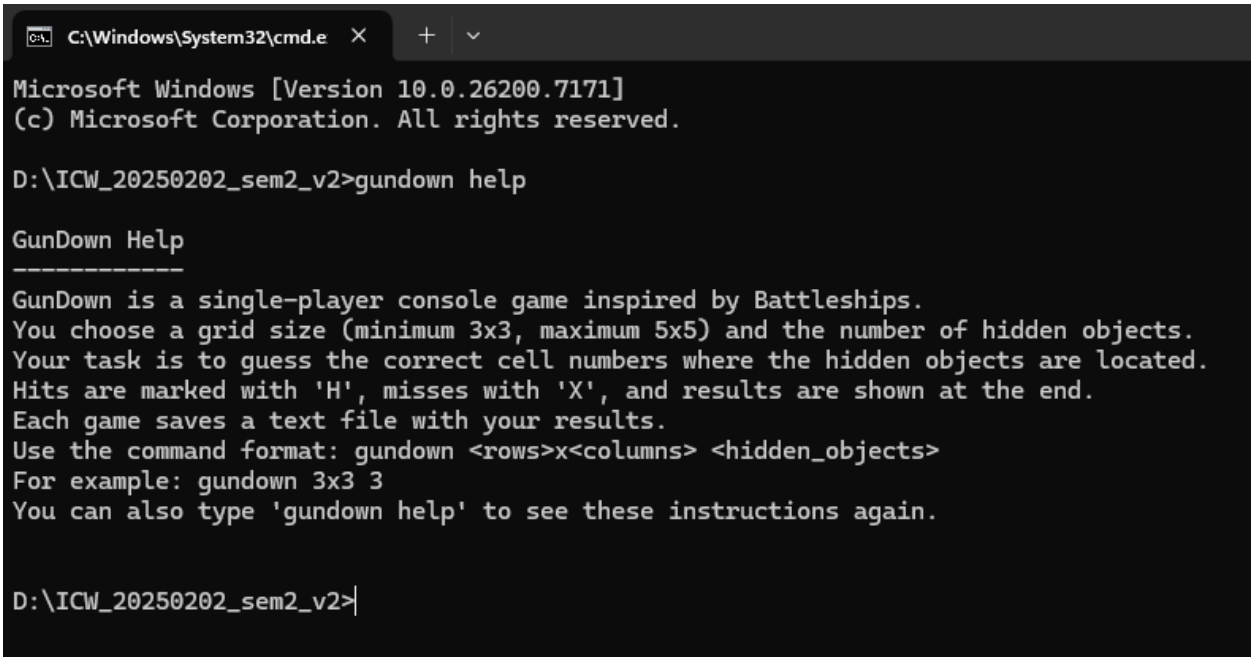


Figure 8 Test Case 2

Test Case 3

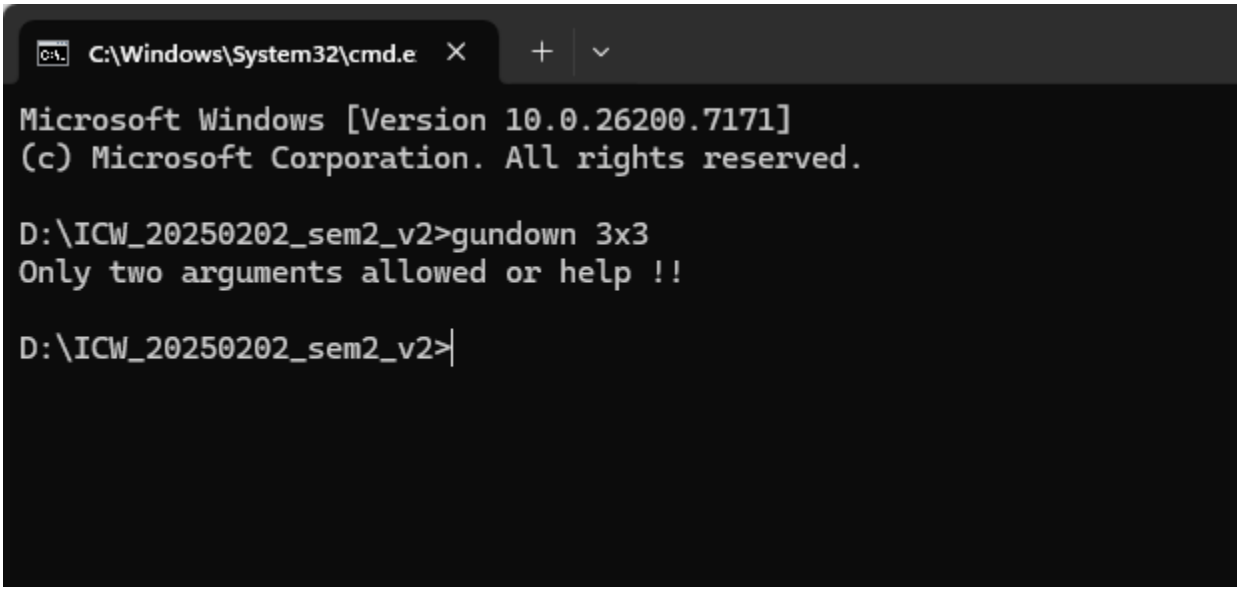


Figure 9 Test Case 3

Test Case 4

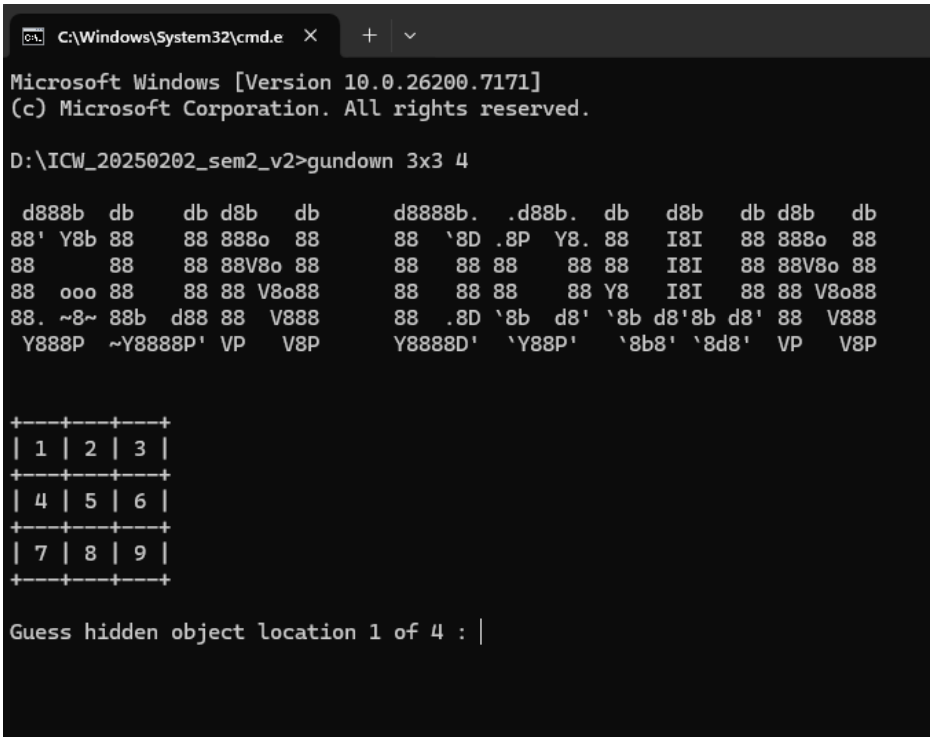


Figure 10 Test Case 4

Test Case 5

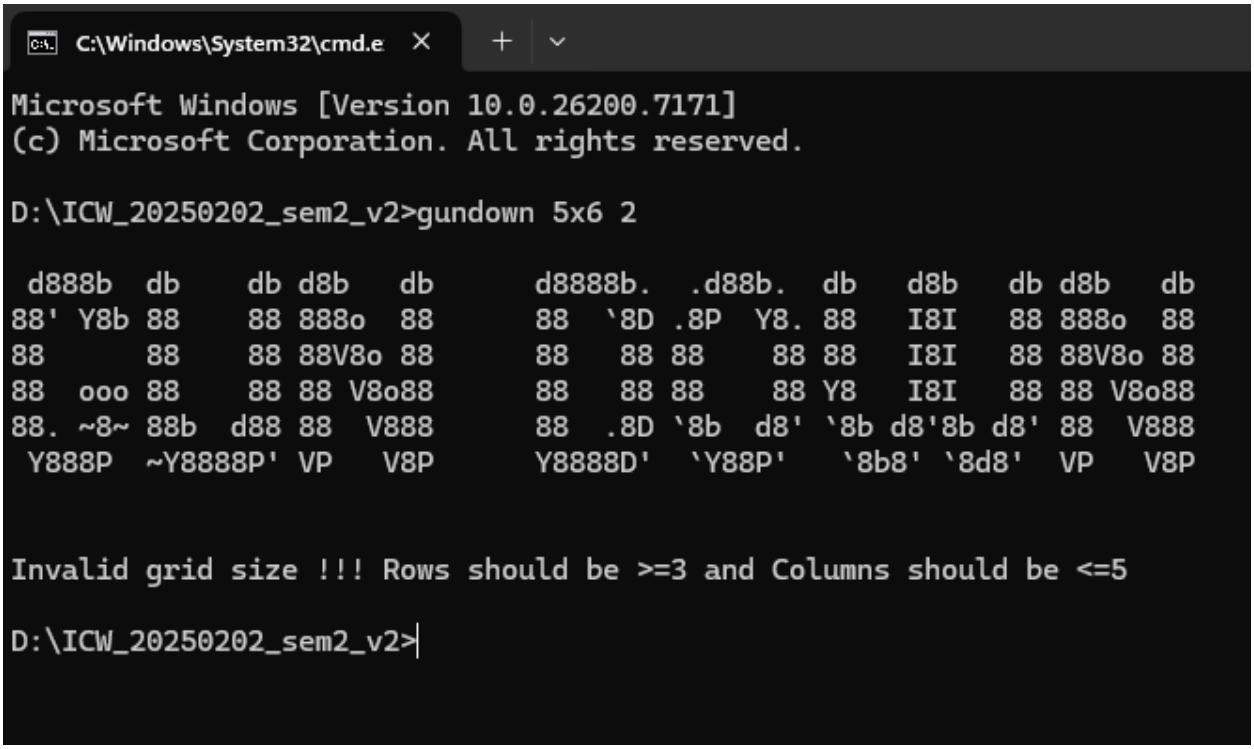


Figure 11 Test Case 5

Test Case 6

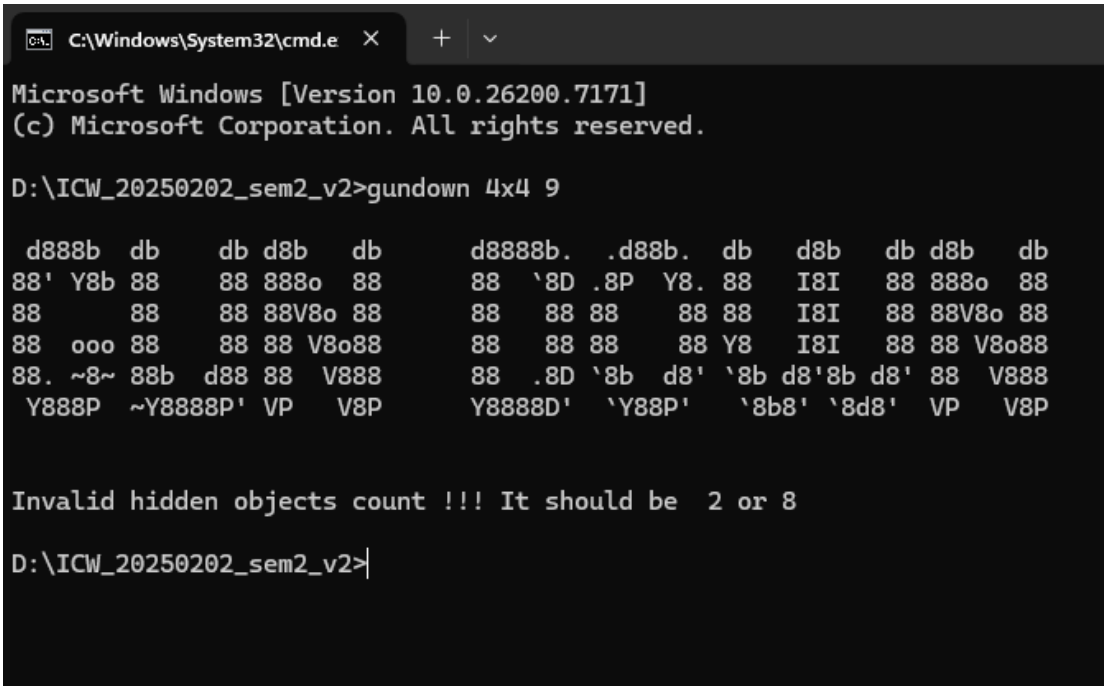


Figure 12 Test Case 6

Test Case 7

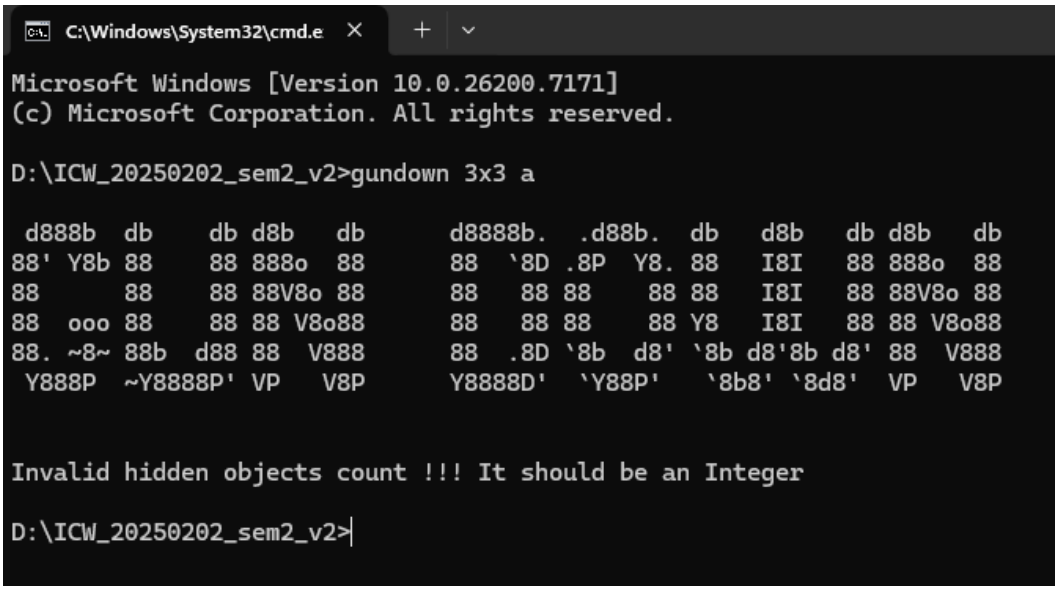


Figure 13 Test Case 7

Test Case 8

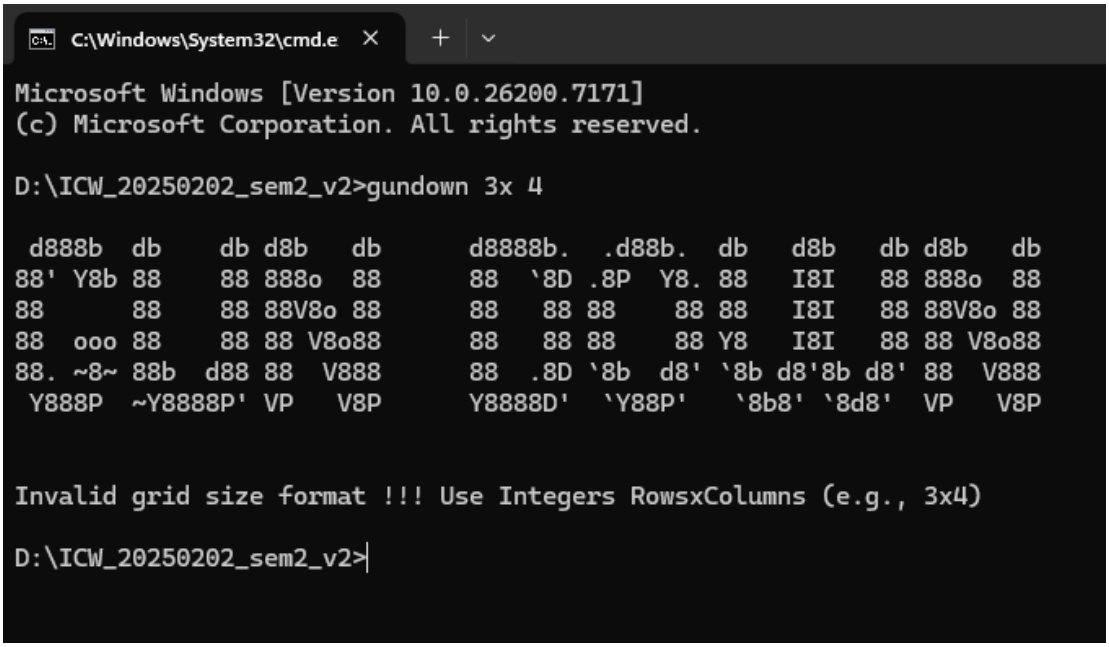


Figure 14 Test Case 8

Test Case 9

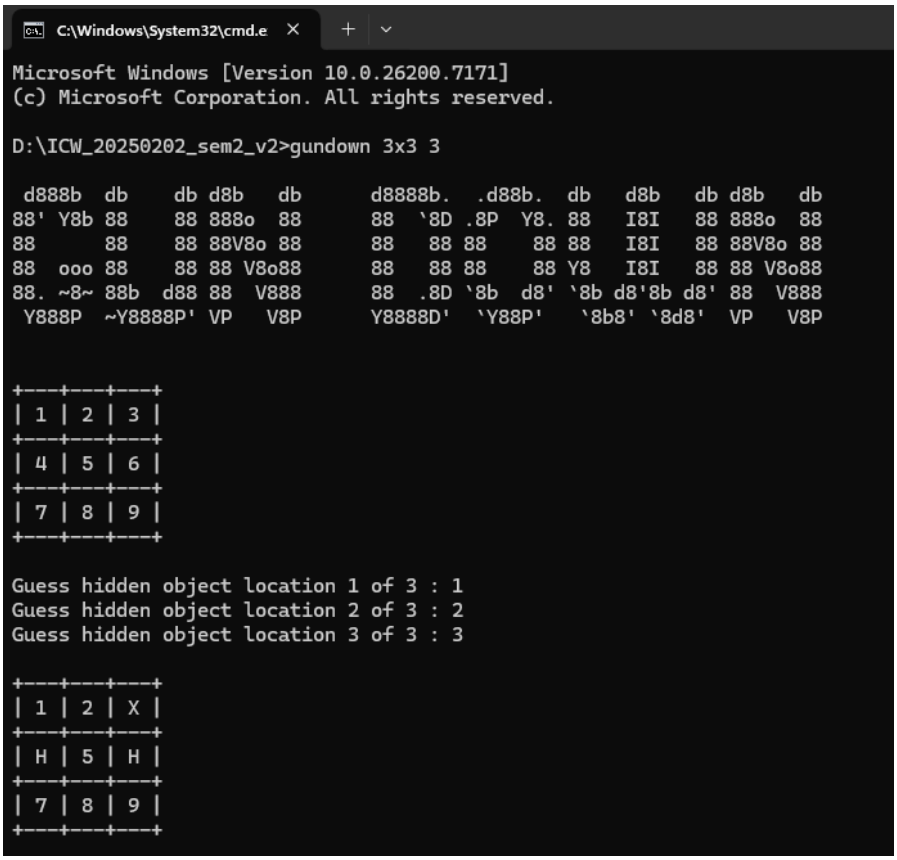


Figure 15 Test Case 9

Test Case 10

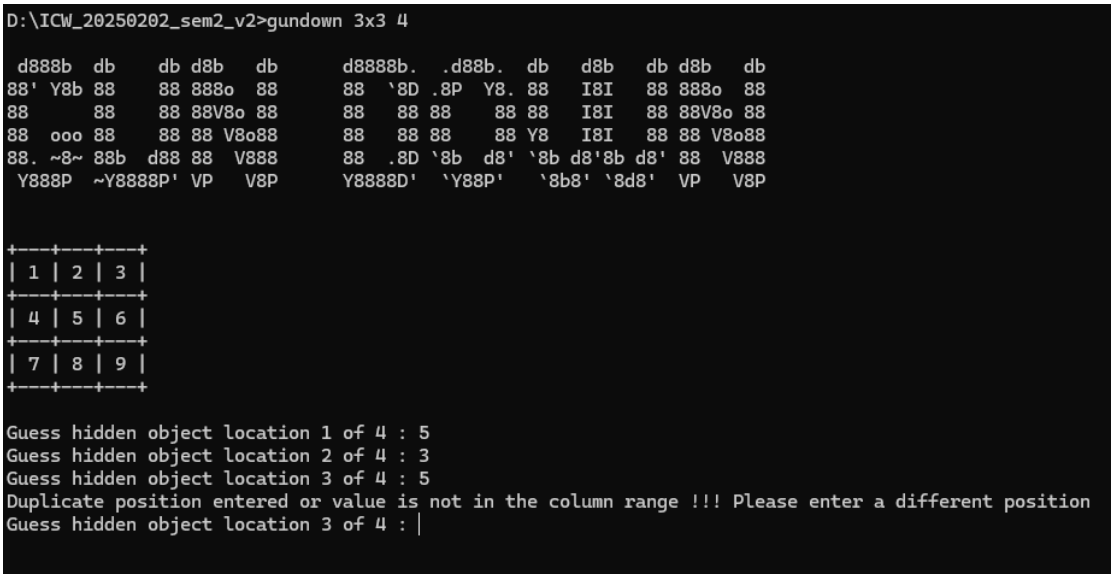


Figure 16 Test Case 10

Test Case 11



Figure 17 Test Case 11

Test Case 12

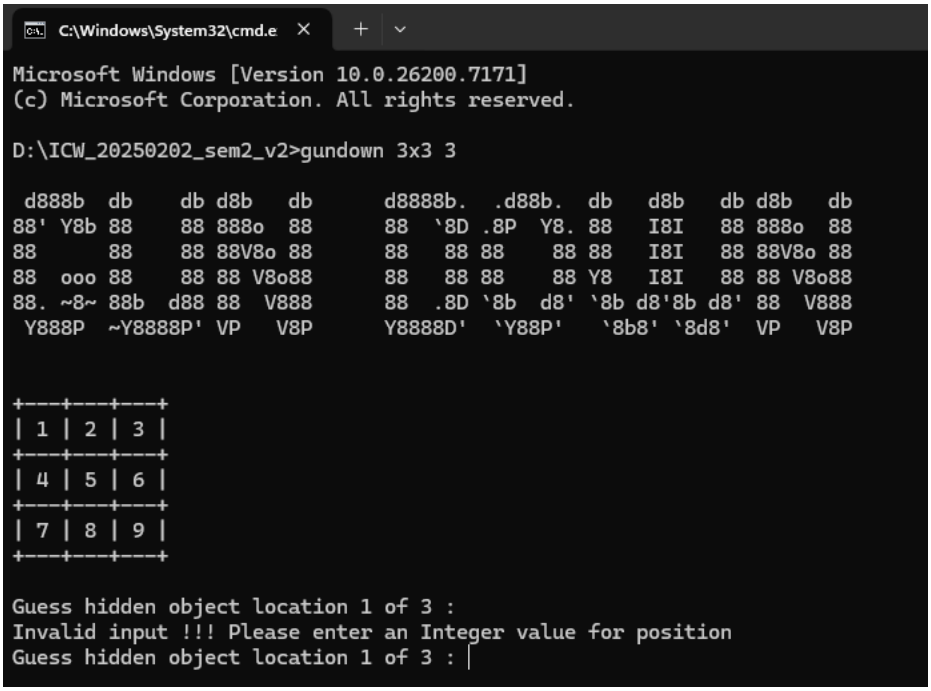


Figure 18 Test Case 12

Test Case 13

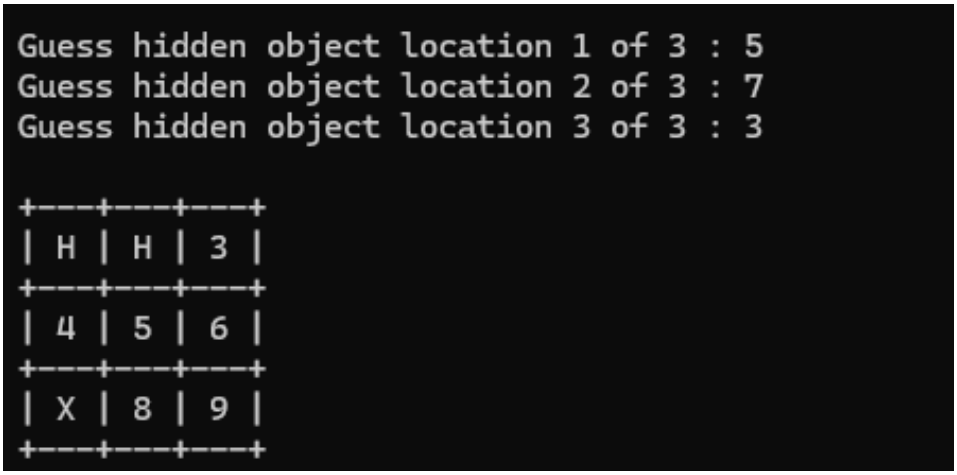


Figure 19 Test Case 13

Test Case 14

```
Guess hidden object location 1 of 3 : 5
Guess hidden object location 2 of 3 : 7
Guess hidden object location 3 of 3 : 3

+---+---+---+
| H | H | 3 |
+---+---+---+
| 4 | 5 | 6 |
+---+---+---+
| X | 8 | 9 |
+---+---+---+

Final Results
Hidden locations : ,2,1,7
Found objects    : ,7
1  of  3 found (33%)
```

Figure 20 Test Case 14

Test Case 15

```
+---+---+---+
| H | X | 3 |
+---+---+---+
| 4 | H | 6 |
+---+---+---+
| 7 | H | 9 |
+---+---+---+

Final Results
Hidden locations : ,5,8,2,1
Found objects    : ,2
1  of  4 found (25%)
```

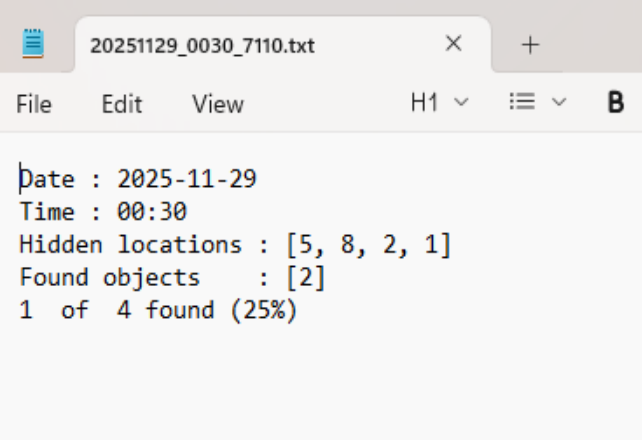


Figure 21 Test Case 15

Test Case 16

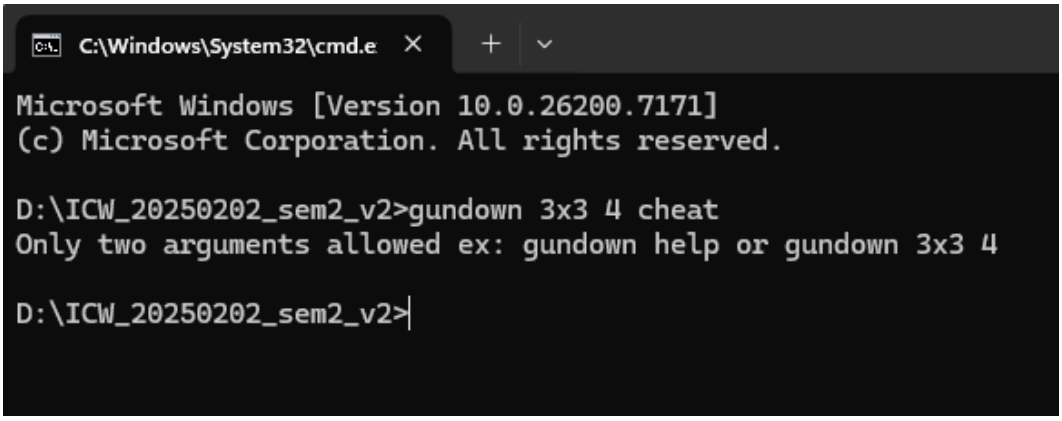


Figure 22 Test Case 16

Test Case 17

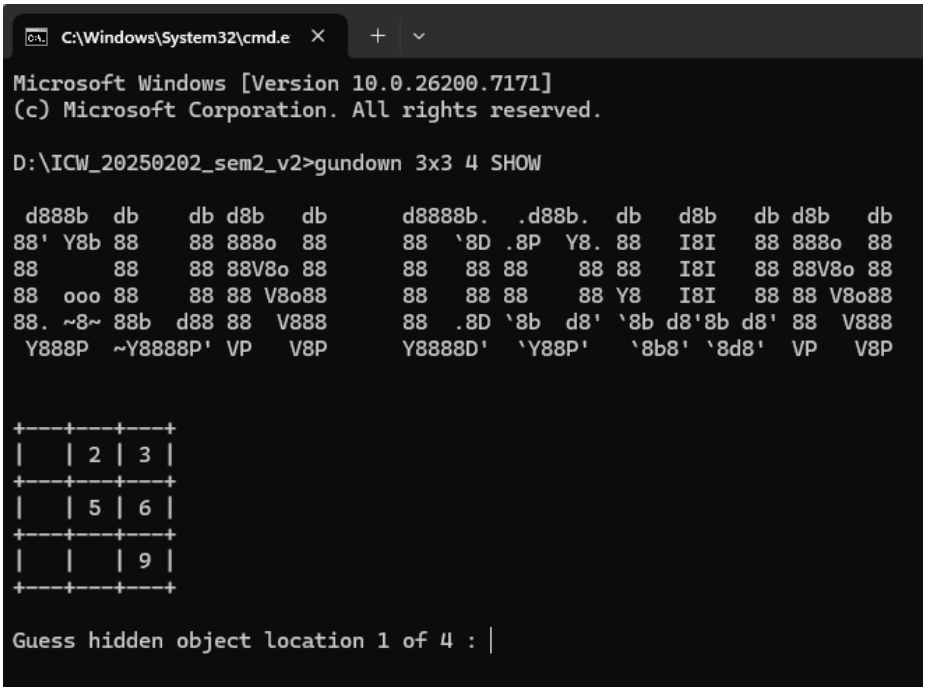


Figure 23 Test Case 17

Test Case 18

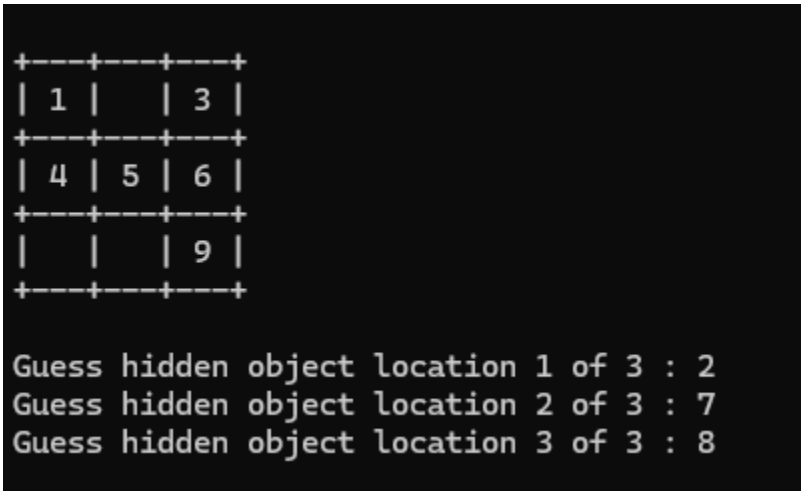
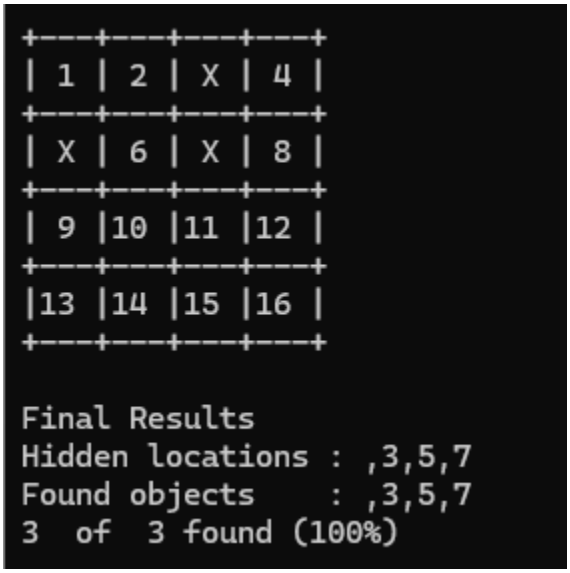


Figure 24 Test Case 18

Test Case 19



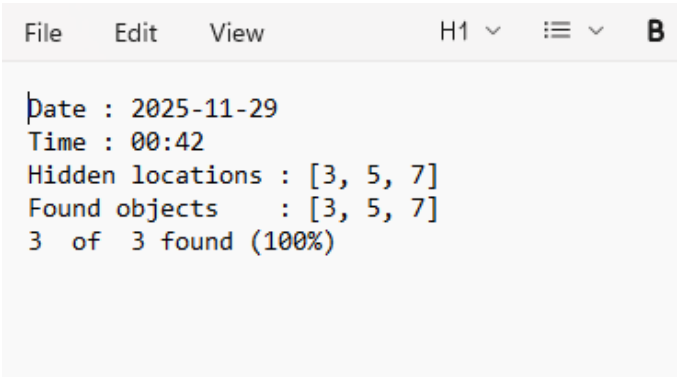


Figure 25 Test Case 19

Test Case Table

Test Case No.	Input data	Expected Output	Actual Output	Result
1	gundown	Display an argument error that is it doesn't have the expected input arguments	Displaying only two arguments allowed or help which means It didn't get the expected arguments	Pass
2	gundown help	Display the help menu extracted from the text file	Display the help menu	Pass
3	gundown 3x3	If the user didn't enter the correct argument count it gives an error message in this case the hidden object count is missing	Display an error that the argument count is not as expected	Pass
4	gundown 3x3 4	If the user inputs are valid then the program execute	As expected	Pass
5	gundown 5x6 2	If the user input incorrect order that is not in range (3<=rows and 5>= columns) then the program show an error that the values are not in valid range	Program gives an error the order range is not valid	Pass
6	gundown 4x4 9	If the user input invalid hidden object count then the program gives as error	Display error invalid hidden object count	Pass
7	gundown 3x3 a	If the user input a non-numeric character or string for the hidden object	Program display and error of about the input is not an integer	Pass

		count then the program shows an error		
8	gundown 3x 4	If user missed a value in the matrix order then program shows an error	Display an error that the gride format is invalid	Pass
9	gundown 3x3 3 guess values : 1,2,3	User will able to input the guess location after completing the user validation	As expected	Pass
10	gundown 3x3 4 guess values : 5,3,5	If the user input the same guess value several times the program display an error and again ask for a correct input	Display error duplicated positions entered	Pass
11	gundown 3x3 3 guess values : a	If user input string for a guess value the program shows an error that the value is not a integer	Display an error that the value is not an integer and ask for enter a positive number that in range of the column count	Pass
12	gundown 3x3 3 guess values :	If User mistakenly skip a guess position grogram display an error that the input is invalid	As expected	Pass
13	gundown 3x3 3 guess values : 5,7,3	Program display the (found hidden values in X and missed hidden values in H)	Display the found locations as X and missed locations as H In a matrix	Pass
14	gundown 3x3 3 guess values : 5,7,3	Display the hidden locations, found locations and the score percentage in the final result	As expected	Pass
15	gundown 3x3 4 guess values :2,4,6,3	If the program show the final results it saves as a text file	Create a text file with date,time,random 4 digit code and saves the final results	Pass
16	gundown 3x3 4 cheat	Display an error if the cheat mode is invalid	Display an error the cheat mode is invalid	Pass
17	gundown 3x3 4 SHOW	If user entered the correct cheat mode argument as show	As expected	Pass

		program execute in cheat mode		
18	gundown 3x3 4 SHOW guess values:2,7,8	User will see the hidden locations in the gride	As expected	Pass
19	gundown 3x3 4 SHOW guess values:3,5,7	The final result show with found hidden locations as X and missed hidden locations as H and the overall score and save it into a text file	Display the final results and create a file with date, time , 4 digit random code and “C” in the text file	Pass

References

1. Anon., 2025. *Geeks for Geeks*. [Online]
   Available at: https://www.geeksforgeeks.org/python/python-random-module/
   [Accessed 28 Nov 2025].

2. Anon., n.d. *W3Schools*. [Online]
   Available at: https://www.w3schools.com/python/python_string_formatting.asp
   [Accessed 28 Nov 2025].